Uninformed Search on Klotski puzzle

Computational Intelligence — Laboratory 4
Politecnico di Torino
s359220@studenti.polito.it

***Abstract*—This report presents comparison of approaches used for solving Klotski puzzle. Several searching methods compared based on search cost and path length. The best-performing search algorithm is further analyzed.**

I. Problem Overview

Klotski is a sliding-block puzzle in which differently sized blocks must be moved within a confined grid to guide a designated target block to the exit. The objective is to find a valid sequence of moves minimizing the number of moves required to reach the goal state.

A search space can be deterministically defined by an initial state. Search strategies differ in how they order the frontier of nodes to be explored.

4 search methods were implemented over 6 problem instances. Focusing on (i) search cost, (ii) the number of moves.

II. Proposed Approach

A. Algorithm Selection

Several algorithms were evaluated:

- BFS search.
- Bi-directional BFS search.
- DFS search.
- DLS + Iterative Deepening.

III. Results

TABLE I
Solution Path Length (Moves)

Solver	P1	P2	P3	P4	P5	P6
BFS	8	18	45	116	179	–
Bi-BFS	8	18	45	116	179	–
DFS	9	18	45	118	187	–
DLS+IDDFS	9	18	45	116	179	–

IV. Discussion and Conclusion

Final selection was toward BFS search. It produces shortest theoretical path and in our state space we encounter our first GOAL state around middle of the tree, it means we can avoid exploring full tree while getting shortest path.

References

[1] PyTorch Documentation, *PyTorch*. Available: https://pytorch.org/docs/
[2] Scikit-learn Documentation, *Scikit-learn*. Available: https://scikit-learn.org/stable/

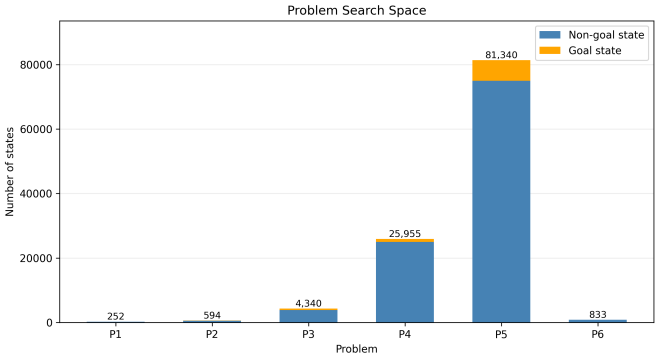


Fig. 1. Enter Caption

Fig. 1 We can see search space with respect to solution space. In problem 6, there is no possible path for reaching solution from given initial state.

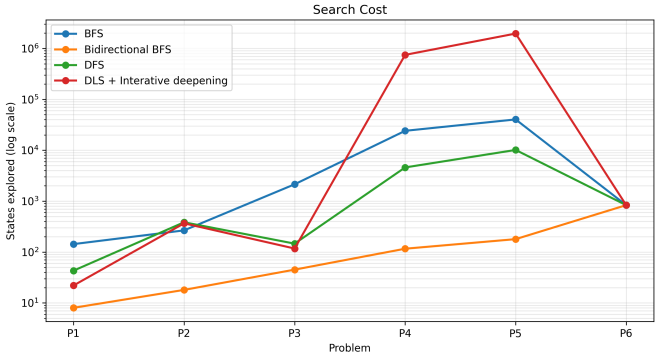


Fig. 2. Models comparison

Fig. 2 We can see 2 main trends. First Bi-directional BFS having always smallest path, it happens because when searching backward from GOAL state, in tree definition we have only 1 parent associated with a node. Secondly with DLS+ID we see that for problems where even first GOAL state is hidden in deep layers, DLS will iteratively increase depth and re-run the whole tree again and again, exploring same path+new node many times

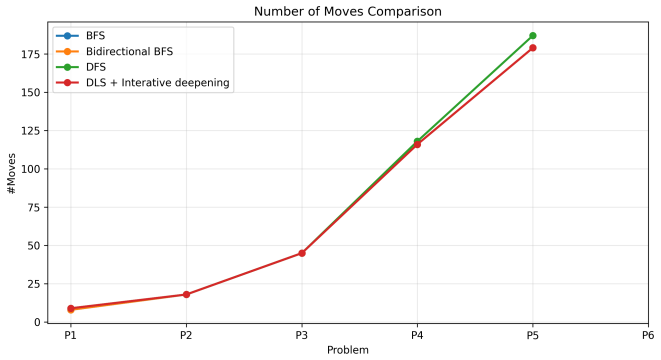


Fig. 3. Solution path length comparison across algorithms.

Fig. 3 shows that BFS and bidirectional BFS produce identical path lengths, while DFS finds longer solutions for P4 and P5.

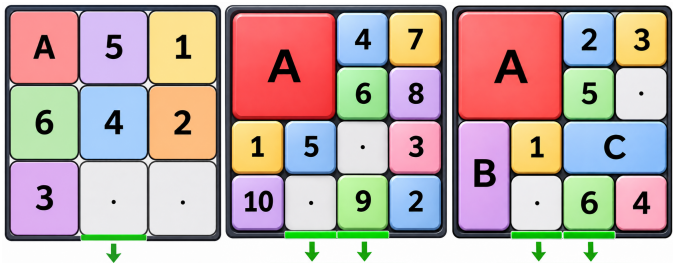


Fig. 4. Problems 1, 2, 3

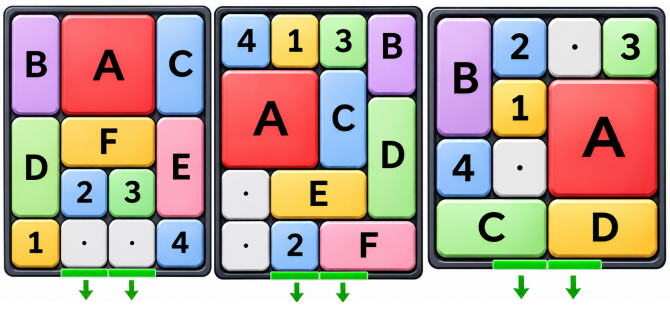


Fig. 5. Problems 1, 2, 3

```python
def bfs_search(json_path):
    with open(json_path, "r") as f:
        data = json.load(f)

    tree = data["tree"]
    states = data["states"]
    goals = set(data["goal_nodes"])

    queue = deque([0])
    visited = {0}
    parent = {0: None}
```

```python
    while queue:
        node = queue.popleft()
        # Check goal
        if node in goals:
            path = []
            current = node

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            print("\n\nGoal found:", node)
            print("Shortest path:", len(path) - 1, "moves")
            print("Nodes explored:", len(visited))

            print("Path (node IDs):")
            print(" -> ".join(map(str, path)))
            return [
                np.array(states[str(i)], dtype=int)
                for i in path
            ]
        # Explore children from saved tree
        for child in tree.get(str(node), []):
            child = int(child)
            if child not in visited:
                visited.add(child)
                parent[child] = node
                queue.append(child)
    print("No solution found")
    return []
```